

系统开发工具基础实验报告

姓名：管恒奕

学号：25020007029

实验环境：Ubuntu Linux (VMware Workstation 虚拟机)

实验时间：2026-08-24

1 实验目的

1. 掌握 Linux 下批量文件处理，学会安全处理包含空格的文件名，使用 `find`、`cp -parents` 完成带目录结构的文件复制，学会批量修改文件与目录权限。
2. 编写 Shell 脚本，实现 CSV 日志文件参数接收、文件存在性判断、`awk` 数据统计，区分标准输出与标准错误输出。
3. 理解 Git 分支开发工作流，复现文件合并冲突，掌握手动解决冲突的流程，读懂带图的提交日志。
4. 掌握 LaTeX 基础文档编写，使用公式、表格、交叉引用，理解 `pdflatex` 编译流程与辅助文件作用，修复排版错误生成 PDF。

2 实验环境

- 操作系统：Ubuntu (虚拟机 VMware Workstation)
- 工具：bash shell、find、cp、chmod、git、pdflatex、nano 文本编辑器

3 实验内容、步骤与结果分析

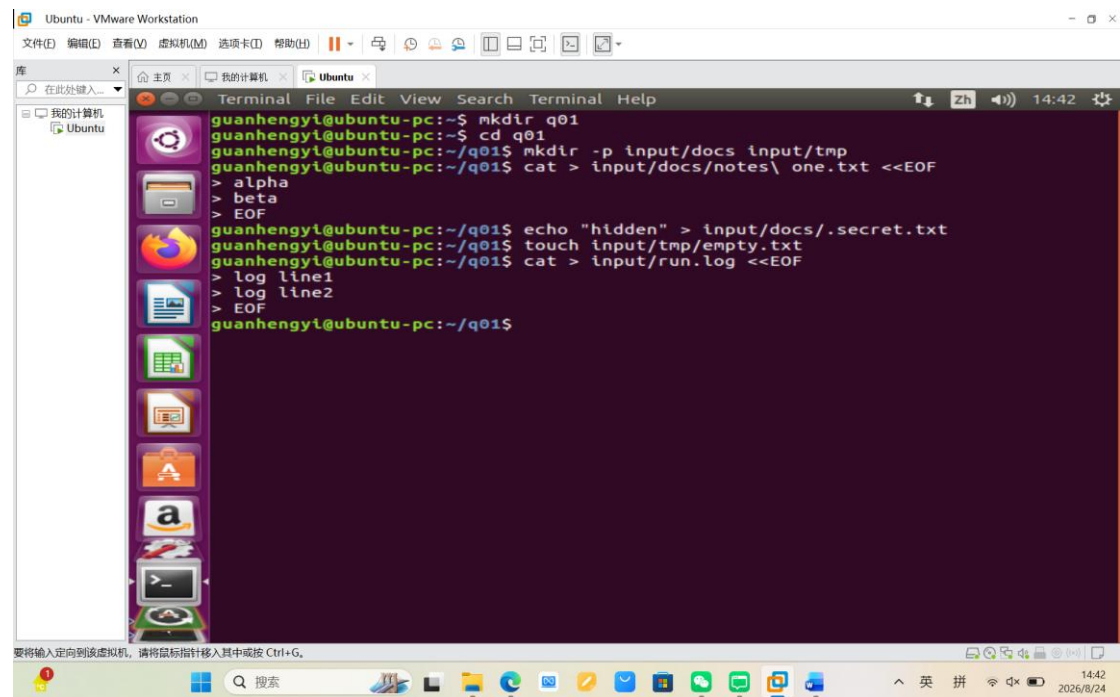
题目一：含空格文件名的批量整理

3.1.1 实验任务

- 创建 `input/docs` 和 `input/tmp`；创建 `input/docs/notes one.txt` (内容为 `alpha` 和 `beta` 两行)、`input/docs/.secret.txt` (内容为 `hidden`)、`input/tmp/empty.txt` (空文件) 以及 `input/run.log` (任意两行日志)。
- 显示 `q01` 的绝对路径，并以长格式列出 `input` 下全部项目，包含隐藏文件。
- 把 `input` 下所有 `.txt` 文件复制到 `work//`，保留相对目录结构并正确处理名称中的空格
- 将复制后的目录权限设为 `750`、普通文件权限设为 `640`；生成 `inventory.txt`，列出相对路径和字节数。

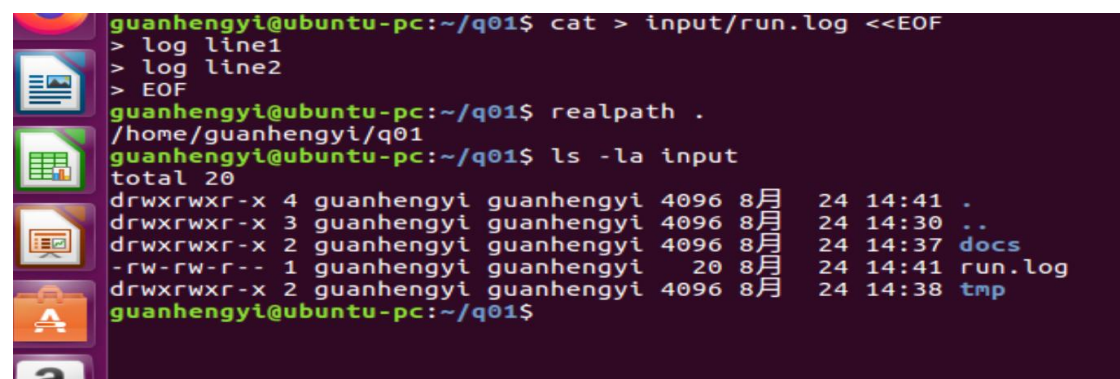
3.1.3 具体步骤

第一步：创建各个目录、文件



```
guanhengyi@ubuntu-pc:~$ mkdir q01
guanhengyi@ubuntu-pc:~/q01$ cd q01
guanhengyi@ubuntu-pc:~/q01$ mkdir -p input/docs input/tmp
guanhengyi@ubuntu-pc:~/q01$ cat > input/docs/notes\ one.txt <<EOF
> alpha
> beta
> EOF
guanhengyi@ubuntu-pc:~/q01$ echo "hidden" > input/docs/.secret.txt
guanhengyi@ubuntu-pc:~/q01$ touch input/tmp/empty.txt
guanhengyi@ubuntu-pc:~/q01$ cat > input/run.log <<EOF
> log line1
> log line2
> EOF
guanhengyi@ubuntu-pc:~/q01$
```

第二步：显示 q01 绝对路径 pwd / realpath .



```
guanhengyi@ubuntu-pc:~/q01$ cat > input/run.log <<EOF
> log line1
> log line2
> EOF
guanhengyi@ubuntu-pc:~/q01$ realpath .
/home/guanhengyi/q01
guanhengyi@ubuntu-pc:~/q01$ ls -la input
total 20
drwxrwxr-x 4 guanhengyi guanhengyi 4096 8月 24 14:41 .
drwxrwxr-x 3 guanhengyi guanhengyi 4096 8月 24 14:30 ..
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:37 docs
-rw-rw-r-- 1 guanhengyi guanhengyi 20 8月 24 14:41 run.log
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:38 tmp
guanhengyi@ubuntu-pc:~/q01$
```

第三步：复制 input 下所有.txt 到 work/<学号>/，保留目录结构，处理空格文件名

```
guanhengyi@ubuntu-pc: ~/q01
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:38 tmp
guanhengyi@ubuntu-pc:~/q01$ mkdir -p work/<25020007029>
bash: syntax error near unexpected token 'newline'
guanhengyi@ubuntu-pc:~/q01$ mkdir -p work/25020007029
guanhengyi@ubuntu-pc:~/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007029/ \;
guanhengyi@ubuntu-pc:~/q01$ ls -laR work/25020007029
work/25020007029:
drwxrwxr-x 3 guanhengyi guanhengyi 4096 8月 24 14:53 .
drwxrwxr-x 3 guanhengyi guanhengyi 4096 8月 24 14:50 ..
drwxrwxr-x 4 guanhengyi guanhengyi 4096 8月 24 14:53 input
work/25020007029/input:
total 16
drwxrwxr-x 4 guanhengyi guanhengyi 4096 8月 24 14:53 .
drwxrwxr-x 3 guanhengyi guanhengyi 4096 8月 24 14:53 ..
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:53 docs
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:53 tmp
work/25020007029/input/docs:
total 16
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:53 .
drwxrwxr-x 4 guanhengyi guanhengyi 4096 8月 24 14:53 ..
-rw-rw-r-- 1 guanhengyi guanhengyi 11 8月 24 14:53 notes one.txt
-rw-rw-r-- 1 guanhengyi guanhengyi 7 8月 24 14:53 .secret.txt
work/25020007029/input/tmp:
total 8
drwxrwxr-x 2 guanhengyi guanhengyi 4096 8月 24 14:53 .
drwxrwxr-x 4 guanhengyi guanhengyi 4096 8月 24 14:53 ..
-rw-rw-r-- 1 guanhengyi guanhengyi 0 8月 24 14:53 empty.txt
guanhengyi@ubuntu-pc:~/q01$
```

第四步：设置权限：目录 750，普通文件 640

```
guanhengyi@ubuntu-pc:~/q01$ cd work/25020007029
guanhengyi@ubuntu-pc:~/q01/work/25020007029$ find . type d -exec chmod 750 {} \;
find: 'type': No such file or directory
find: 'd': No such file or directory
guanhengyi@ubuntu-pc:~/q01/work/25020007029$ find . -type d -exec chmod 750 {} \;
guanhengyi@ubuntu-pc:~/q01/work/25020007029$ find . -type f -exec chmod 640 {} \;
guanhengyi@ubuntu-pc:~/q01/work/25020007029$ cd ../../
guanhengyi@ubuntu-pc:~/q01$

guanhengyi@ubuntu-pc:~/q01$ find work/25020007029 -type f -printf "%P %s\n" > inventory.txt
guanhengyi@ubuntu-pc:~/q01$
```

生成 inventory.txt:

```
guanhengyi@ubuntu-pc:~/q01$ cat inventory.txt
input/docs/notes one.txt 11
input/docs/.secret.txt 7
input/tmp/empty.txt 0
guanhengyi@ubuntu-pc:~/q01$
```

3.1.4 问题与排错

报错: bash: syntax error near unexpected token 'newline'

- 错误原因: find -exec 结尾的; 标记被换行打断, -exec 必须以; 作为结束标记, 不能随意换行。
- 解决: 保证; 和前面命令在逻辑连续, 不随意拆分换行。

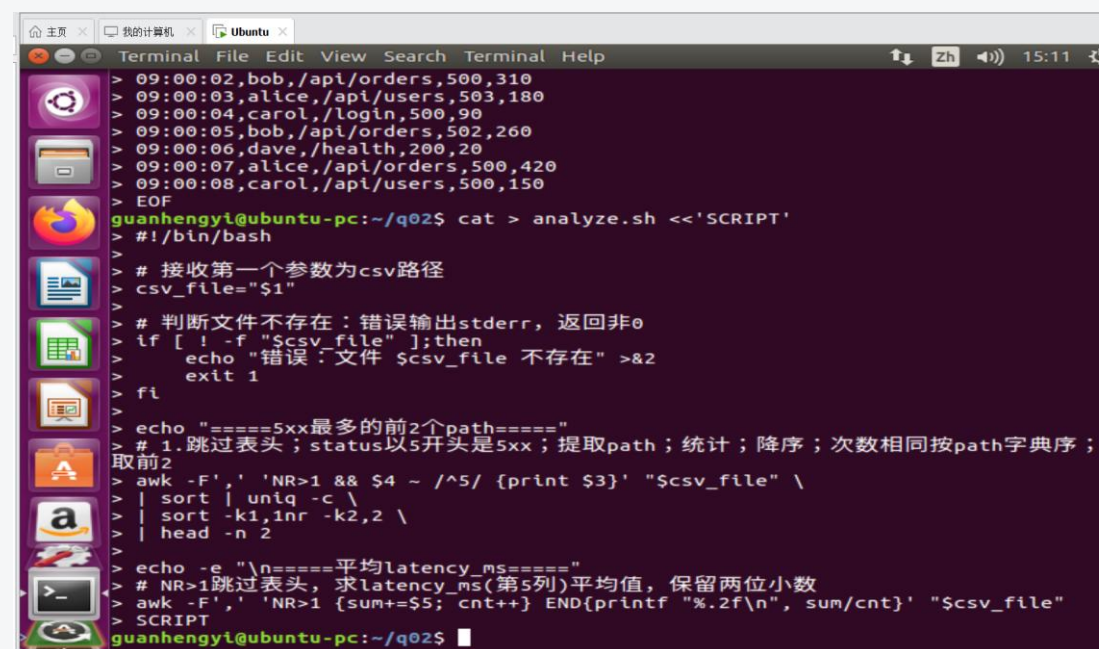
题目二：随机访问日志统计

3.2.1 实验任务

1. 创建 `q02` 目录，准备 `access.csv` 访问日志。
2. 编写 `analyze.sh` 脚本：接收 CSV 路径作为参数；判断文件是否存在，不存在输出错误信息到标准错误；统计状态码 `5xx` 出现次数最多的前 2 个访问路径；计算延迟字段 `latency_ms` 平均值，保留两位小数。
3. 给脚本添加执行权限，分别对存在文件、不存在文件执行脚本，观察输出与退出码。

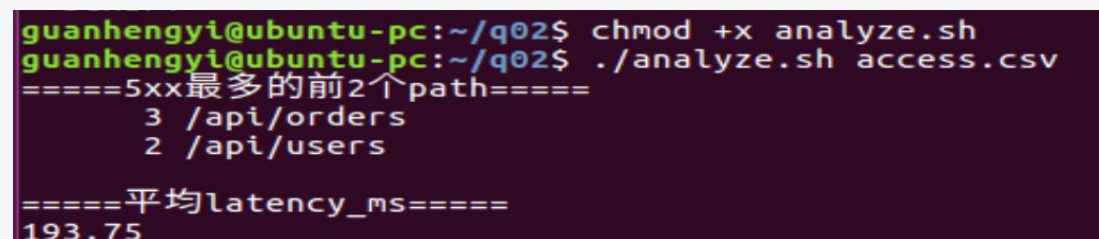
3.2.2 具体步骤

第一步 创建 `q02` 文件夹，生成 `access.csv` 编写脚本 `analyze.sh`



```
Terminal File Edit View Search Terminal Help
> 09:00:02,bob,/api/orders,500,310
> 09:00:03,alice,/api/users,503,180
> 09:00:04,carol,/login,500,90
> 09:00:05,bob,/api/orders,502,260
> 09:00:06,dave,/health,200,20
> 09:00:07,alice,/api/orders,500,420
> 09:00:08,carol,/api/users,500,150
> EOF
guanhengyi@ubuntu-pc:~/q02$ cat > analyze.sh <<'SCRIPT'
#!/bin/bash
# 接收第一个参数为csv路径
csv_file="$1"
# 判断文件不存在：错误输出stderr，返回非0
if [ ! -f "$csv_file" ];then
    echo "错误：文件 $csv_file 不存在" >&2
    exit 1
fi
echo "====5xx最多的前2个path===="
# 1.跳过表头；status以5开头是5xx；提取path；统计；降序；次数相同按path字典序；取前2
awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}' "$csv_file" \
| sort | uniq -c \
| sort -k1,1nr -k2,2 \
| head -n 2
echo -e "\n====平均latency_ms===="
# NR>1跳过表头，求latency_ms(第5列)平均值，保留两位小数
awk -F',' 'NR>1 {sum+=$5; cnt++} END{printf "%.2f\n", sum/cnt}' "$csv_file"
SCRIPT
guanhengyi@ubuntu-pc:~/q02$
```

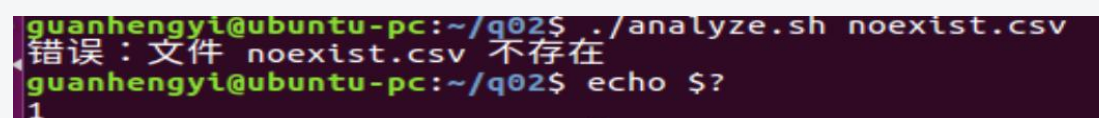
①给脚本加上执行权限 对存在的 `access.csv` 运行



```
guanhengyi@ubuntu-pc:~/q02$ chmod +x analyze.sh
guanhengyi@ubuntu-pc:~/q02$ ./analyze.sh access.csv
====5xx最多的前2个path====
    3 /api/orders
    2 /api/users

====平均latency_ms====
193.75
```

②对不存在文件运行



```
guanhengyi@ubuntu-pc:~/q02$ ./analyze.sh noexist.csv
错误：文件 noexist.csv 不存在
guanhengyi@ubuntu-pc:~/q02$ echo $?
1
```


3.2.3 关键知识点解释

脚本逻辑说明

1. `[ ! -f "$csv_file" ]` 判断文件不存在；`>&2` 把消息输出到标准错误 `stderr`；`exit 1` 返回非 0 退出码，满足题目错误要求。
2. **5xx 统计管道拆解**
 - `awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}'`：跳过表头，status 第 4 列以 5 开头，输出 path 字段
 - `sort | uniq -c`：排序，统计每个 path 出现次数
 - `sort -k1,1nr -k2,2`：第一列（次数）**数字降序**；次数相同，第二列 (path) **字典升序**
 - `head -n 2`：取前 2 行

`"$1"` 变量加双引号：防止文件名包含空格时发生参数切割。

`>&2`：把普通 `echo` 文本重定向到标准错误 `stderr`；错误信息不和正常输出混杂。

`awk -F','` 指定逗号作为字段分隔符；`NR>1` 跳过 CSV 表头行。

`exit 1`：异常场景返回非 0 退出码，用于外部程序判断脚本执行是否出错。

题目三：制造、解决并解释一次合并冲突

3.3.1 实验任务

1. 在 `main` 分支创建 `config.txt`，做初始提交。
2. 创建 `feature-a` 分支，修改 `config.txt` 后提交。
3. 切回 `main` 初始提交，新建 `feature-b` 分支，修改同一处内容提交。
4. 返回 `main` 分支，先合并 `feature-a`，再合并 `feature-b`，触发合并冲突。
5. 解决冲突：保留 `mode=safe`，新增一行 `note=reviewed`，删除冲突标记，完成提交。
6. 工作区干净，执行 `git log --all --graph --decorate --oneline` 查看分支提交图。

3.3.2 完整操作命令

第一步：main 分支创建 config.txt，初始提交

```
guanhengyi@ubuntu-pc:~$ mkdir q03
guanhengyi@ubuntu-pc:~$ cd q03
guanhengyi@ubuntu-pc:~/q03$ git init
Initialized empty Git repository in /home/guanhengyi/q03/.git/
guanhengyi@ubuntu-pc:~/q03$ echo "mode=normal" > config.txt
guanhengyi@ubuntu-pc:~/q03$ git add config.txt
guanhengyi@ubuntu-pc:~/q03$ git commit -m "init: config mode=normal"
[master (root-commit) 6d49f58] init: config mode=normal
1 file changed, 1 insertion(+)
create mode 100644 config.txt
```

第二步：创建 feature-a 分支，修改并提交

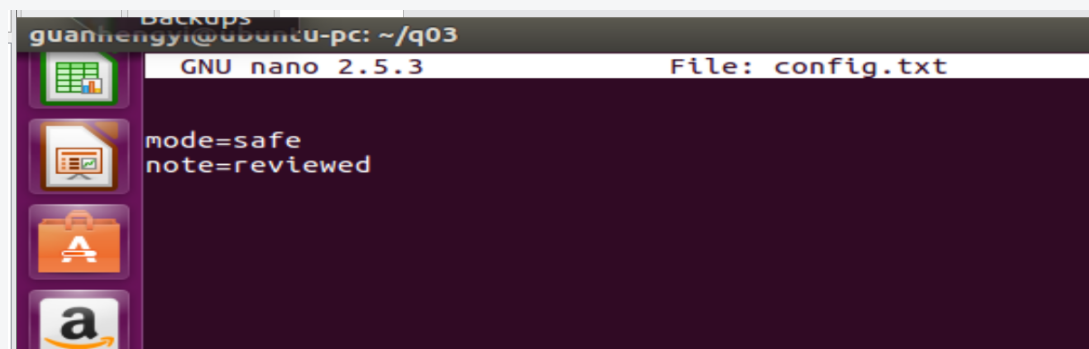
```
guanhengyi@ubuntu-pc:~/q03$ git checkout -b feature-a
Switched to a new branch 'feature-a'
guanhengyi@ubuntu-pc:~/q03$ echo "mode=safe" > config.txt
guanhengyi@ubuntu-pc:~/q03$ git add config.txt
guanhengyi@ubuntu-pc:~/q03$ git commit -m "feature-a: set mode=safe"
[feature-a df103f4] feature-a: set mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)
guanhengyi@ubuntu-pc:~/q03$
```

第三步：切回初始 main 提交点，新建 feature-b 分支

```
guanhengyi@ubuntu-pc:~/q03$ git branch -M main
guanhengyi@ubuntu-pc:~/q03$ git branch
  feature-a
* main
guanhengyi@ubuntu-pc:~/q03$ git checkout main
Already on 'main'
guanhengyi@ubuntu-pc:~/q03$ git checkout -b feature-b
Switched to a new branch 'feature-b'
guanhengyi@ubuntu-pc:~/q03$ echo "mode=fast" > config.txt
guanhengyi@ubuntu-pc:~/q03$ git add config.txt
guanhengyi@ubuntu-pc:~/q03$ git commit -m "feature-b: set mode=fast"
[feature-b 1fc86d5] feature-b: set mode=fast
1 file changed, 1 insertion(+), 1 deletion(-)
guanhengyi@ubuntu-pc:~/q03$
```

第四步：回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed

```
guanhengyi@ubuntu-pc:~/q03$ git checkout main
Switched to branch 'main'
guanhengyi@ubuntu-pc:~/q03$ git merge feature-a
Updating 6d49f58..df103f4
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
guanhengyi@ubuntu-pc:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
guanhengyi@ubuntu-pc:~/q03$
```



```
Backups
guanhengyi@ubuntu-pc: ~/q03
GNU nano 2.5.3                               File: config.txt

mode=safe
note=reviewed
```

第五步：确认工作区干净，运行 `git log --all --graph --decorate --oneline`

```
Automatic Merge failed; fix conflicts and then commit the
guanhengyi@ubuntu-pc:~/q03$ nano config.txt
guanhengyi@ubuntu-pc:~/q03$ git add config.txt
guanhengyi@ubuntu-pc:~/q03$ git commit
[main 67ae8fb] Merge branch 'feature-b' into main
guanhengyi@ubuntu-pc:~/q03$ git status
On branch main
nothing to commit, working directory clean
guanhengyi@ubuntu-pc:~/q03$ git log --all --graph --decorate --oneline
* 67ae8fb Merge branch 'feature-b' into main
|
| * 1fc86d5 feature-b: set mode=fast
| * | df103f4 feature-a: set mode=safe
|/
* 6d49f58 init: config mode=normal
guanhengyi@ubuntu-pc:~/q03$
```

3.3.4 结果分析

1. 冲突产生原因：两个分支 `feature-a` 与 `feature-b` 修改了同一个文件的同一行文本，Git 无法自动判断选用哪一份修改，需要人工介入处理。
2. `git log --all --graph --decorate --oneline` 输出中：*代表提交节点，竖线、斜线展示分支分叉与合并关系，合并操作会产生一个全新的合并提交。
3. 冲突解决完成后，`git status` 输出 `nothing to commit, working directory clean` 代表工作区无未提交修改。

题目四：修复并构建一页 LaTeX 技术说明 PDF

3.4.1 实验任务

1. 创建 `report.tex`，编写 LaTeX 文档，包含标题、章节、公式、表格、交叉引用。
2. 使用 `pdflatex` 编译生成 PDF，排查表格排版异常问题。
3. 理解 `\label` 与 `\ref` 交叉引用，了解 `aux` 辅助文件，需要多次编译更新引用编号。

3.4.2 实验步骤

第一步：创建目录与文件

```
guanhengyi@ubuntu-pc: ~/q04
guanhengyi@ubuntu-pc:~$ mkdir q04
guanhengyi@ubuntu-pc:~$ cd q04
guanhengyi@ubuntu-pc:~/q04$ nano report.tex
guanhengyi@ubuntu-pc:~/q04$
```

```
GNU nano 2.5.3 File: report.tex

\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{guanhengyi-25020007029}
\maketitle

\section{Result}

下面展示公式\eqref{eq:energy}，这是一个能量计算公式。
\begin{equation}
E = mc^2
\label{eq:energy}
\end{equation}

表\ref{tab:data}展示两组实验数据。
\begin{table}[htbp]
\centering
\caption{实验数据表}
\label{tab:data}
\begin{tabular}{cc}
A & B \\
10 & 20
\end{tabular}
\end{table}

公式\ref{eq:energy}与表\ref{tab:data}为本次主要结果。
```

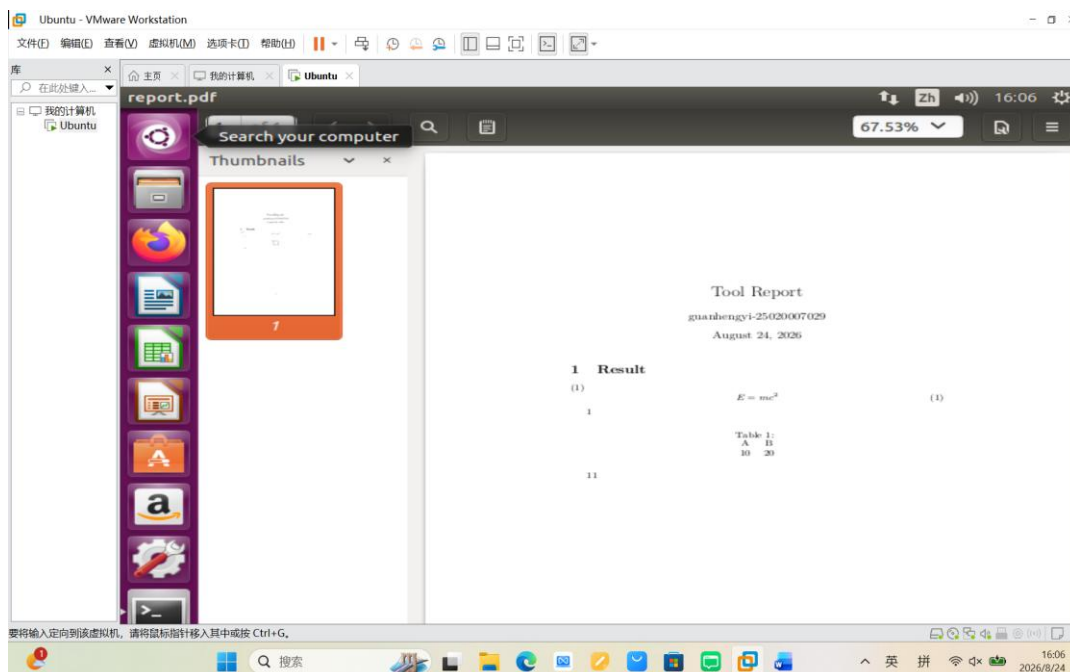
第二步：编译生成 PDF

```
guanhengyi@ubuntu-pc:~/q04$ latexmk -pdf -halt-on-error report.tex
```

检查结果：

```
guanhengyi@ubuntu-pc:~/q04$ ls
report.aux  report.fdb_latexmk  report.fls  report.log  report.pdf  report.tex
guanhengyi@ubuntu-pc:~/q04$
```

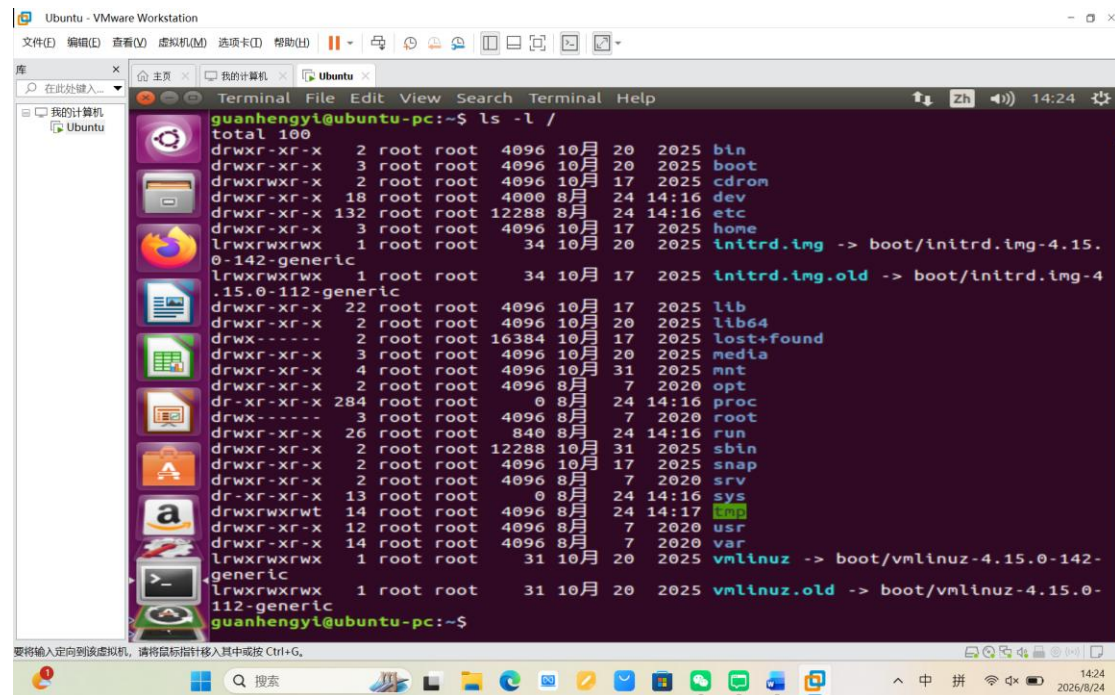
pdf 文件打开：xdg-open report.pdf



4 课后实例

课程概览 + Shell 入门

(1) `ls` 的 `-l` 选项 (flag) 作用是什么？运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么？



```
guanhengyi@ubuntu-pc:~$ ls -l /
total 100
drwxr-xr-x  2 root root  4096 10月 20 2025 bin
drwxr-xr-x  3 root root  4096 10月 20 2025 boot
drwxrwxr-x  2 root root  4096 10月 17 2025 cdrom
drwxr-xr-x 18 root root  4000 8月 24 14:16 dev
drwxr-xr-x 132 root root 12288 8月 24 14:16 etc
drwxr-xr-x  3 root root  4096 10月 17 2025 home
lrwxrwxrwx  1 root root   34 10月 20 2025 initrd.img -> boot/initrd.img-4.15.0-142-generic
lrwxrwxrwx  1 root root   34 10月 17 2025 initrd.img.old -> boot/initrd.img-4.15.0-112-generic
drwxr-xr-x 22 root root  4096 10月 17 2025 lib
drwxr-xr-x  2 root root  4096 10月 20 2025 lib64
drwx----- 2 root root 16384 10月 17 2025 lost+found
drwxr-xr-x  3 root root  4096 10月 20 2025 media
drwxr-xr-x  4 root root  4096 10月 31 2025 mnt
drwxr-xr-x  2 root root  4096 8月 7 2020 opt
dr-xr-xr-x 284 root root    0 8月 24 14:16 proc
drwx----- 3 root root  4096 8月 7 2020 root
drwxr-xr-x 26 root root   840 8月 24 14:16 run
drwxr-xr-x  2 root root 12288 10月 31 2025/sbin
drwxr-xr-x  2 root root  4096 10月 17 2025/snap
drwxr-xr-x  2 root root  4096 8月 7 2020/srv
dr-xr-xr-x 13 root root    0 8月 24 14:16/sys
drwxrwxrwt 14 root root  4096 8月 24 14:17/tmp
drwxr-xr-x 12 root root  4096 8月 7 2020/usr
drwxr-xr-x 14 root root  4096 8月 7 2020/var
lrwxrwxrwx  1 root root   31 10月 20 2025/vmlinuz -> boot/vmlinuz-4.15.0-142-generic
lrwxrwxrwx  1 root root   31 10月 20 2025/vmlinuz.old -> boot/vmlinuz-4.15.0-112-generic
guanhengyi@ubuntu-pc:~$
```

答：`ls -l` 的 `-l` (long, 长格式)：以长列表格式显示文件 / 目录的完整详细信息，不只是文件名，会输出权限、文件类型、硬链接数、所有者、所属组、大小、修改时间、文件名。

第 1 位文件类型 `d` = directory 目录；`-` = 普通文件；`l`=link 软链接（符号链接）；`b/c` 块 / 字符设备（`dev` 目录里）

2-4 位属主（owner，文件拥有者）权限

5-7 位属组（group，所属用户组）权限

8-10 位其他用户（others，其他人）权限

(2) '单引号'、"双引号" 和 `$'ANSI 引号'` 有什么区别？写一条命令，输出一个同时包含字面量 `$`、`!` 和换行符的字符串。

```
guanhengyi@ubuntu-pc: ~  
guanhengyi@ubuntu-pc:~$ echo '$ \n !'  
$ \n !  
guanhengyi@ubuntu-pc:~$ echo "$ \n !"  
$ \n !  
guanhengyi@ubuntu-pc:~$ echo $'美元符号：$ \n感叹号：!'  
美元符号：$  
感叹号：!  
guanhengyi@ubuntu-pc:~$
```

单引号强引用，所有符号全部失去特殊意义

双引号弱引用，半解析大部分字符原样，但是：

- `$`、```会生效：变量会展开
- 交互 shell 中 `!` 会触发历史命令，经常报错 `event not found`
- `\n` 不会自动翻译成换行

`$'` ANSI-C 引号 不做变量展开：`$` 就是普通字符

1. 解析 C 语言转义字符：`\n` 换行、`\t` 制表、`\\` 反斜杠
2. `!` 直接作为普通字符，不会触发历史命令

(3) `$?` 保存上一条命令的退出状态（0 表示成功）。`&&` 仅在前一条成功时执行后一条；`||` 仅在前一条失败时执行后一条。写一个一行命令：仅当 `/tmp/mydir` 不存在时才创建它。

```
guanhengyi@ubuntu-pc:~$ test -d /tmp/mydir || mkdir /tmp/mydir  
guanhengyi@ubuntu-pc:~$
```

(4) 写一条命令，把文件复制为带当天日期的备份文件名（例如 `notes.txt` $\rightarrow$ `notes_2026-01-12.txt`）。（提示：`$(date +%Y-%m-%d)`）

```
guanhengyi@ubuntu-pc:~$ echo "test content" > notes.txt  
guanhengyi@ubuntu-pc:~$ cp notes.txt notes_$(date +%Y-%m-%d).txt  
guanhengyi@ubuntu-pc:~$ ls notes*.txt  
notes_2026-08-26.txt notes.txt  
guanhengyi@ubuntu-pc:~$
```

(5) 在命令 `find ~/Downloads -type f -name "*.zip" -mtime +30` 中，`*.zip` 是一个「glob」。什么是 glob？新建一个测试目录并创建一些文件，试试 `ls *.txt`、`ls file?.txt`、`ls {a,b,c}.txt` 等模式。

答：glob（全局通配符，文件名元字符扩展）：Bash shell 的一种文件名匹配机制。shell 在执行命令之前，会把 `*`、`?`、`[]` 这类 glob 模式，自动替换成磁盘上匹配的真实文件名，再交给程序执行。

```

guanhengyi@ubuntu-pc:~$ mkdir ~/glob_test
guanhengyi@ubuntu-pc:~$ cd ~/glob_test
guanhengyi@ubuntu-pc:~/glob_test$ touch a.txt b.txt c.txt file1.txt file2.txt fi
lex.txt file.txt hello.txt
guanhengyi@ubuntu-pc:~/glob_test$ ls *.txt
a.txt b.txt c.txt file1.txt file2.txt file.txt filex.txt hello.txt
guanhengyi@ubuntu-pc:~/glob_test$ ls file?.txt
file1.txt file2.txt filex.txt
guanhengyi@ubuntu-pc:~/glob_test$ ls {a,b,c}.txt
a.txt b.txt c.txt
guanhengyi@ubuntu-pc:~/glob_test$ █

```

(6) 使用管道找出你「home 目录」中最常见的 5 种文件扩展名。

```

guanhengyi@ubuntu-pc:~$ find ~ -type f | sed -E 's/.*\.[^\.]+$/\1/' | grep -v '
/' | sort | uniq -c | sort -nr | head -n 5
    101 gz
     30 pb
     21 txt
     18 vlpset
     18 sbstore
guanhengyi@ubuntu-pc:~$ █

```

Version Control and Git

(7) One common mistake when learning Git is to commit large files that should not be managed by Git or adding sensitive information. Try adding a file to a repository, making some commits and then deleting that file from *history* (not just the latest commit).

题目翻译

学习 Git 时一个常见错误是提交了本应提交的大文件 不能被 Git 管理，也不会添加敏感信息。试着添加一个文件到一个仓库，先提交一些文件，然后从历史中删除该文件（不仅仅是最新的提交）。

```

guanhengyi@ubuntu-pc:~$ mkdir test_git
guanhengyi@ubuntu-pc:~$ cd test_git
guanhengyi@ubuntu-pc:~/test_git$ git init
Initialized empty Git repository in /home/guanhengyi/test_git/.git/
guanhengyi@ubuntu-pc:~/test_git$ echo "sensitive content here" > badfile.txt
guanhengyi@ubuntu-pc:~/test_git$ git add badfile.txt
guanhengyi@ubuntu-pc:~/test_git$ git commit -m "commit1 add badfile.txt"
[master (root-commit) 4a74967] commit1 add badfile.txt
 1 file changed, 1 insertion(+)
 create mode 100644 badfile.txt
guanhengyi@ubuntu-pc:~/test_git$ echo "normal text" > a.txt
guanhengyi@ubuntu-pc:~/test_git$ git add a.txt
guanhengyi@ubuntu-pc:~/test_git$ git commit -m "commit2 add a.txt"
[master de1d163] commit2 add a.txt
 1 file changed, 1 insertion(+)
 create mode 100644 a.txt
guanhengyi@ubuntu-pc:~/test_git$ echo "hello world" > b.txt
guanhengyi@ubuntu-pc:~/test_git$ git add b.txt
guanhengyi@ubuntu-pc:~/test_git$ git commit -m "commit3 add b.txt"
[master 2695504] commit3 add b.txt
 1 file changed, 1 insertion(+)
 create mode 100644 b.txt

guanhengyi@ubuntu-pc:~/test_git$ git filter-branch --force --index-filter "git r
m --cached --ignore-unmatch badfile.txt" --prune-empty -- --all
Rewrite 4a74967217c3caa9147adbd81fb6bd1e60652395 (1/3) (0 seconds passed, remain
ing 0 predicted)    rm 'badfile.txt'
Rewrite de1d163ba0da4f0d20f6e38b8b374679c1c252a3 (2/3) (0 seconds passed, rema
ing 0 predicted)    rm 'badfile.txt'
Rewrite 2695504a98597305ed268af0aa2b457902917dfa (3/3) (0 seconds passed, remain
ing 0 predicted)    rm 'badfile.txt'

Ref 'refs/heads/master' was rewritten
guanhengyi@ubuntu-pc:~/test_git$ git log -- badfile.txt
guanhengyi@ubuntu-pc:~/test_git$ █

```


(8) Clone some repository from GitHub, and modify one of its existing files. What happens when you do ? What do you see when running ? Run to undo what you did with . In what scenario might this be useful?git stashgit log --all --onelinegit stash popgit stash

题目翻译

从 GitHub 上克隆某个仓库，修改其现有文件。当你真的去了会怎样？跑步时你看到了什么？跑去撤销你之前做的事。在什么情况下这会有用？git stashgit log --all --onelinegit stash popgit stash

```
guanhengyi@ubuntu-pc:~$ git clone https://github.com/octocat/Hello-World.git
Cloning into 'Hello-World'...
remote: Enumerating objects: 13, done.
remote: Total 13 (delta 0), reused 0 (delta 0), pack-reused 13 (from 1)
Unpacking objects: 100% (13/13), done.
Checking connectivity... done.
guanhengyi@ubuntu-pc:~$ cd Hello-World
guanhengyi@ubuntu-pc:~/Hello-World$ echo "my local change test" >> README
guanhengyi@ubuntu-pc:~/Hello-World$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   README

no changes added to commit (use "git add" and/or "git commit -a")
guanhengyi@ubuntu-pc:~/Hello-World$ git stash
Saved working directory and index state WIP on master: 7fd1a60 Merge pull request #6 from Spaceghost/patch-1
HEAD is now at 7fd1a60 Merge pull request #6 from Spaceghost/patch-1
guanhengyi@ubuntu-pc:~/Hello-World$ git log --all --oneline
30b3441 WIP on master: 7fd1a60 Merge pull request #6 from Spaceghost/patch-1
25bb79e index on master: 7fd1a60 Merge pull request #6 from Spaceghost/patch-1
b1b3f97 sentence case
b3cbd5b Create CONTRIBUTING.md
7fd1a60 Merge pull request #6 from Spaceghost/patch-1
7629413 New line at end of file. --Signed off by Spaceghost
553c207 first commit
guanhengyi@ubuntu-pc:~/Hello-World$ git stash pop
```

- 答：1. 修改文件后，改动只保存在本地工作区，没有提交。执行`git stash`，会把未提交的修改临时存放到 stash 堆栈，本地工作目录恢复成干净状态。
2. `git log --all --oneline`除普通提交记录外，可以看到一条特殊的`stash@{0}`临时保存记录。
3. `git stash pop`将暂存的改动恢复到工作区，同时删除这条 stash 记录。
4. 适用场景：正在写半成品代码，不想生成临时提交，但需要切换分支或者拉取远程代码，先用`git stash`把改动存起来，处理完别的工作后，再用`git stash pop`恢复继续开发。

(9) Like many command line tools, Git provides a configuration file (or dotfile) called . Create an alias in so that when you run , you get the output of . You can do this by directly [editing](#) the file, or you can use the command to add the alias.

题目翻译

和很多命令行工具一样，Git 有一个配置文件（点文件）`~/.gitconfig`。在 `~/.gitconfig` 中创建别名：执行 `git graph`，等价于执行 `git log --all --graph --decorate --oneline`。可以直接编辑该配置文件，也可以用 `git config` 命令添加别名。

```
guanhengyi@ubuntu-pc:~$ git config --global alias.graph "log --all --graph --decorate --oneline"
guanhengyi@ubuntu-pc:~$ git graph
fatal: Not a git repository (or any of the parent directories): .git
guanhengyi@ubuntu-pc:~$ cd Hello-World
guanhengyi@ubuntu-pc:~/Hello-World$ git graph
* b1b3f97 (origin/octocat-patch-1) sentence case
* b3cbd5b (origin/test) Create CONTRIBUTING.md
* 7fd1a60 (HEAD -> master, origin/master, origin/HEAD) Merge pull request #6 from Spaceghost/patch-1
* 7629413 New line at end of file. --Signed off by Spaceghost
* 553c207 first commit
guanhengyi@ubuntu-pc:~/Hello-World$ git config --global --get alias.graph
log --all --graph --decorate --oneline
guanhengyi@ubuntu-pc:~/Hello-World$
guanhengyi@ubuntu-pc:~/Hello-World$
```

(10) You can define global ignore patterns in after running `git config --global core.excludesfile ~/.gitignore_global`. This sets the location of the global ignore file that Git will use, but you still need to manually create the file at that path. Set up your global gitignore file to ignore OS-specific or editor-specific temporary files, like `~/.gitignore_global`.

题目翻译

可以配置全局忽略规则文件 `~/.gitignore_global`，需要先执行 `git config --global core.excludesfile ~/.gitignore_global`。这条命令告诉 Git 全局忽略文件的位置，但文件本身需要你手动新建。配置全局 `.gitignore_global`，忽略系统、编辑器产生的临时文件，例如 `.DS_Store`。

```
guanhengyi@ubuntu-pc:~$ git config --global core.excludesfile ~/.gitignore_global
guanhengyi@ubuntu-pc:~$ cat > ~/.gitignore_global <<EOF
> .DS_Store
> .DS_Store?
> *
> *~
> *.swp
> *.swo
> EOF
guanhengyi@ubuntu-pc:~$ git config --global --get core.excludesfile
/home/guanhengyi/.gitignore_global
guanhengyi@ubuntu-pc:~$
```

(11) You can define global ignore patterns in after running `git config --global core.excludesfile ~/.gitignore_global`. This sets the location of the global ignore file that Git will use, but you still need to manually create the file at that path. Set up your global gitignore file to ignore OS-specific or editor-specific temporary files, like `~/.gitignore_global`.

题目翻译

你可以在 `~/.gitignore_global` 中定义全局忽略规则，需要先执行命令 `git config --`

global core.excludesfile ~/.gitignore_global。该命令告诉 Git 全局忽略文件的位置，但需要手动创建这个文件。配置你的全局 gitignore 文件，忽略操作系统、编辑器生成的临时文件，例如 .DS_Store。

```
guanhengyi@ubuntu-pc:~$ git config --global core.excludesfile ~/.gitignore_global
guanhengyi@ubuntu-pc:~$ cat > ~/.gitignore_global <<'EOF'
> # macOS系统文件
> .DS_Store
> .DS_Store?
> .*
>
> # 编辑器临时文件
> *.swp
> *.sw0
> *~
> EOF
guanhengyi@ubuntu-pc:~$
guanhengyi@ubuntu-pc:~$ git config --global --get core.excludesfile
/home/guanhengyi/.gitignore_global
guanhengyi@ubuntu-pc:~$
```

5 实验遇到的问题汇总

Q1 含空格文件名的批量整理题

问题 1:

报错: `bash: syntax error near unexpected token 'newline'`

- 错误原因: `find -exec` 结尾的;`标记被换行打断`, `-exec` 必须以;`作为结束标记`, 不能随意换行。
- 解决: 保证;`和前面命令在逻辑连续`, 不随意拆分换行。

Q3 LaTeX Git 题 (q03 git)

问题 1: git 合并冲突, 手动编辑冲突文件

- 现象: `git merge` 出现冲突标记<<<<<<、=====、>>>>>>
- 解决: 打开文件, 删掉冲突标记, 保留正确内容保存; 执行 `git add 文件名`; `git commit` 完成冲突提交。

问题 2: git 分支切换、合并操作, 历史查看

- 现象: 分不清当前在哪个分支; 想看提交记录
- 解决: `git branch` 看所有分支, `git checkout` 分支名切换; `git log` 查看提交历史。

问题 3: git 合并后查看结果验证

- 现象：不知道合并是否成功
- 解决：git status 查看状态，看文件内容确认修改，git log --oneline 看提交链。

Q4 LaTeX q04 report.tex

问题：执行 latexmk 提示 **command not found**, latexmk 未安装

- 现象：'latexmk' is currently not installed
- 解决：下载 latexmk: sudo apt update ; sudo apt install latexmk texlive-base texlive-latex-extra

6 解题感悟

通过这次 Linux 综合实验，我接触了 Shell 脚本、Git 版本控制还有 LaTeX 排版，收获很多。

之前用命令行处理文件的时候，没太在意文件名带空格这种情况，这次实验才发现这是很容易出错的地方。直接拼接路径很容易被空格拆分，要用 find 结合 {} 或者给变量加双引号才能正常处理。权限设置也不能图省事直接递归修改，目录和普通文件的权限要求不一样，要分开设置。写 shell 脚本也让我明白，脚本不能只考虑正常运行，还要做判断，比如判断文件存不存在，区分普通输出和错误输出，返回对应的退出码，这样脚本才算完整。

做日志统计的时候，感受到 awk 处理文本的方便。处理 CSV 文件要设置分隔符，还要记得跳过表头，不然统计结果就会出错。写脚本的时候少个引号、漏个符号就直接报错，需要细心检查每一处语法。

在做 Git 合并冲突实验时，我理解了 Git 没办法自动处理所有代码修改。当两个分支改动同一个地方，就会产生冲突，不能只删掉冲突标记就完事，得看懂两边分支改了什么，手动保留需要的内容。同时也学会查看图形化提交日志，还有 git stash 临时保存未完成的代码，这些都是实际写代码时会用到的操作。

LaTeX 和平时用的 Word 差别很大，不是写好直接就能出 PDF。交叉引用需要多次编译生成辅助文件才能正常显示编号，表格的浮动参数也会影响排版。